Indian Institute of Technology Bombay

Department of Electrical Engineering

Design and Verification of a Dual-Clock Asynchronous FIFO

Gray-Coded Pointer Crossing, Assertion-Based Verification and Hardware Validation on PYNQ-Z2

Self Project Report

Submitted by

Jyotishman Deori

Roll No: 25M1186

M.Tech. (Electrical Engineering)

Batch: 2025–27

August 2026

Abstract

Moving data between two clock domains that share no timing relationship is one of the few places in digital design where a circuit can be functionally correct in simulation and still fail in silicon. The standard solution, a FIFO with Gray-coded pointers and two-flop synchronisers, is widely published; what is less often shown is evidence that each mechanism is doing the job attributed to it.

This project implements a parameterised dual-clock asynchronous FIFO in Verilog-2001 and verifies it at four levels: a randomised scoreboard testbench under ModelSim, nine SystemVerilog assertions with functional coverage under Vivado XSim, out-of-context synthesis for the Zynq XC7Z020, and finally execution on a PYNQ-Z2 board. Faults were deliberately seeded into the design before any result was accepted, so that the verification could be shown capable of failing.

Two findings are of interest beyond the design itself. First, removing the Gray coding entirely and crossing raw binary pointers did *not* fail the testbench, because RTL simulation changes every bit of a bus at the same instant and therefore cannot represent the hazard Gray coding exists to prevent. A controlled skew injection experiment was built to expose it, and shows binary pointers corrupting 89 % of transferred words once bus skew exceeds one destination clock period, while Gray coding holds at zero errors throughout. Second, Vivado's CDC report identified a combinational path into a synchroniser's asynchronous clear pin in the author's own hardware test harness, a defect of exactly the class the design was built to avoid.

On hardware, with the write domain clocked from the processing-system PLL and the read domain from the board's independent 125 MHz oscillator, the FIFO transferred **286,176,459** words with **zero** errors while reaching both the full and empty boundaries. Synthesis inferred no latches and produced no warnings.

Contents

List of Figures

List of Tables

1. Introduction

Nearly every non-trivial digital system contains more than one clock. A processor runs at one frequency, an interface at another, a sensor at a third, and data has to pass between them. Wherever two clocks have no fixed phase relationship, the boundary between them is a place where a design can be logically correct and still fail intermittently in hardware for reasons that never appear in simulation.

The asynchronous FIFO is the standard structure used to cross such a boundary. Its construction is well documented, most influentially by Cummings [1], and its two defensive mechanisms are widely quoted: Gray-coded pointers so that a value sampled mid-transition is never a value the counter did not hold, and two-flop synchronisers so that a metastable flip-flop is given a full clock period to resolve.

The motivation for building one from scratch was specific. In my M.Tech seminar work an AXI-Stream interface carried data between the processing system and the programmable logic of a Zynq device, and the clock crossing was handled by a handshake I had not written and could not fully justify. Building the crossing itself, and then attempting to prove that each of its parts earns its place, is a more demanding exercise than instantiating a library FIFO.

1.1 Objectives

The project was organised around five questions:

1. How should a parameterised dual-clock FIFO be structured so that the full and empty conditions remain correct under an arbitrary phase relationship between the two clocks?
2. What verification is sufficient to justify confidence in such a design, and how can that verification itself be shown capable of detecting a defect?
3. Does the Gray coding demonstrably contribute to correctness, or is it accepted on authority?
4. Does the synthesised implementation match the intent, in particular with regard to inferred latches and the tool's recognition of the clock crossings?
5. Does the design behave correctly on real silicon, driven by two genuinely independent oscillators rather than a simulator's idealised clocks?

1.2 Contributions

- A parameterised asynchronous FIFO in Verilog-2001, with the two-flop synchroniser kept as a separate module so that its presence is explicit.
- A randomised scoreboard testbench exercising both clock ratios, with each phase drained so that every word written is compared on the way out.
- Nine SystemVerilog assertions and four covergroups, bound onto the design rather than embedded in it, keeping the synthesisable source free of verification code.

- A fault-injection campaign in which the verification was required to fail before its passing result was accepted.
- A controlled skew-injection experiment quantifying the contribution of the Gray encoding, including the negative result that ordinary RTL simulation cannot distinguish Gray from binary pointers at all.
- Out-of-context synthesis for the Zynq XC7Z020 with zero latches and zero warnings, and interpretation of the resulting CDC report.
- Hardware validation on a PYNQ-Z2 across two independent crystal oscillators: 286,176,459 words transferred with zero errors.

2. Literature Review

2.1 Metastability and Synchronisers

When a flip-flop's setup or hold constraint is violated, its output may enter a metastable state and remain at an indeterminate level for an unbounded time before resolving. Ginosar [2] gives a tutorial treatment and, in a companion paper [3], catalogues the ways designers commonly defeat their own synchronisers. The standard mitigation is the two-flop synchroniser, in which the first flip-flop absorbs the violation and the second samples it only after a full clock period of settling time.

The protection is statistical rather than absolute. Mean time between failures depends exponentially on the settling time allowed, so a synchroniser that is adequate at one frequency may not be at another. Dally and Poulton [4] develop the underlying model. This distinction matters for the present work: the hardware results reported here demonstrate correct operation, but they do not and cannot constitute a measurement of synchroniser reliability.

2.2 Gray Codes for Pointer Crossing

A Gray code [5] is an ordering of binary values in which consecutive entries differ in exactly one bit position. Sampling such a value while it changes therefore yields either the previous value or the next one, never a third. This property is what makes a counter safe to observe from a foreign clock domain, and it is the reason FIFO pointers are converted before crossing. The conversion from binary is a single exclusive-or with a right shift, and the inverse is a prefix exclusive-or.

2.3 Asynchronous FIFO Architectures

Cummings [1] describes the now-standard structure in which each pointer is one bit wider than the memory address, is converted to Gray code, crosses through a two-flop synchroniser, and is compared against the local pointer to produce the full and empty flags. A companion paper [6] develops a variant using asynchronous pointer comparison. The design presented here follows the first structure. The comparison used for the full flag, in which the top two bits of the synchronised pointer are inverted, is a frequent source of error and is examined in detail in Section 4.1.

2.4 Assertion-Based Verification

SystemVerilog Assertions, standardised in IEEE 1800 [7], allow temporal properties to be stated declaratively and checked continuously during simulation. For a CDC design they are particularly appropriate, because the properties of interest, such as "the Gray pointer never changes more than one bit in a cycle", are invariants rather than end-of-test conditions. The `bind` construct permits such checkers to be attached to a design unit externally, so that no verification code appears in the synthesisable source.

3. Background

3.1 The Multi-Bit Crossing Hazard

Consider a binary counter observed from an unrelated clock domain. On the transition from 0111 to 1000 all four bits change. Because each bit follows its own physical path, with its own routing delay, the bits do not arrive simultaneously at the sampling flip-flops. A sample taken inside that window captures a mixture of old and new bits, and the resulting value may be any of the sixteen possible codes rather than either of the two legitimate ones.

Gray coding removes the hazard by construction: consecutive values differ in one bit, so at most one bit is in flight at any moment and the sampled value is necessarily either the old or the new one. Figure 2 illustrates both cases using the actual skew pattern applied in Section 4.5.

The two mechanisms address different failures and neither substitutes for the other. Gray coding does nothing to prevent metastability, since a single bit sampled during its transition can still violate setup. A synchroniser does nothing to prevent multi-bit skew, since it faithfully propagates whatever combination it captured. Omitting either leaves a design that is defective in a manner ordinary simulation may never reveal, a claim tested directly in Section 5.4.

3.2 Design and Verification Environment

Table 1 lists the module interface and Table 2 the tool and platform environment. The FIFO is parameterised in data width and address width; all results reported here use an eight-bit datapath and a depth of sixteen.

Table 1: FIFO module interface

Signal	Dir	Domain	Function
`wclk, wrst_n`	in	write	Write clock and active-low asynchronous reset
`winc`	in	write	Write request; ignored when `wfull` is asserted
`wdata`	in	write	Write data, `DSIZE` bits
`wfull`	out	write	FIFO cannot accept a further word
`rclk, rrst_n`	in	read	Read clock and active-low asynchronous reset
`rinc`	in	read	Read request; ignored when `rempty` is asserted
`rdata`	out	read	Read data, valid whenever `rempty` is low
`rempty`	out	read	FIFO holds no unread word

Table 2: Design and verification environment

Item	Value	Notes
RTL language	Verilog-2001	Compiles without `-sv`
Verification language	SystemVerilog	Testbench and bound checker
Functional simulator	ModelSim ASE 20.1	No SVA or covergroup support
Assertion simulator	Vivado XSim 2020.2	SVA and functional coverage
Synthesis	Vivado 2020.2	Out of context
Target device	xc7z020clg400-1	Zynq-7020, PYNQ-Z2
Write clock (hardware)	100 MHz	FCLK_CLK0, PS PLL, 33.333 MHz crystal
Read clock (hardware)	125 MHz	Board oscillator, separate crystal
FIFO configuration	DSIZE 8, ASIZE 4	Depth 16 words

4. Methodology

4.1 RTL Design

The design comprises two files: `sync_2ff.v`, a parameterised two-flop synchroniser carrying the `ASYNC_REG` attribute, and `async_fifo.v`, containing the memory, the two pointer counters, the Gray conversion and the flag logic. Figure 1 shows the structure.

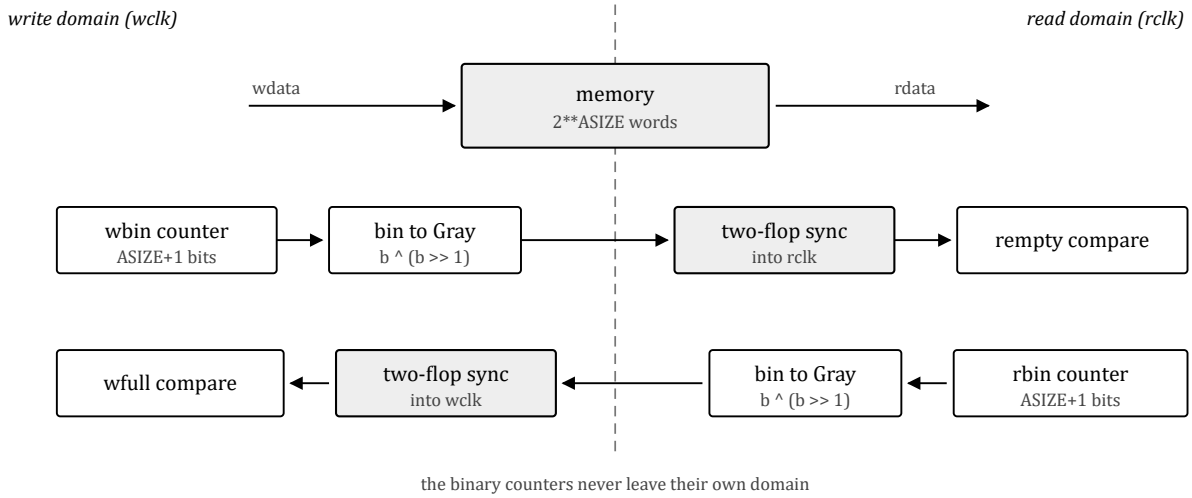

Figure 1: Block diagram of the dual-clock FIFO. Each pointer crosses once, in one direction, as a Gray-coded copy passing through a two-flop synchroniser. The binary counters themselves never cross the boundary.

Pointers are `ASIZE+1` bits wide while the memory address uses only the low `ASIZE` bits. The additional most-significant bit acts as a wrap flag and is the sole means of distinguishing the full condition from the empty condition, since the addresses coincide in both cases. Conversion to Gray code is

```
gray = bin ^ (bin >> 1)
```
(1)

The empty condition is a plain equality: the read pointer has caught the write pointer with no intervening wrap, so every bit agrees.

```
rempty = (rgray_next == rq2_wptr)
```
(2)

The full condition is where the design is most easily got wrong. The write pointer must have wrapped exactly once more than the read pointer and caught it, which in Gray code requires the low bits to agree while the top *two* bits are inverted:

```
wfull = (wgray_next == {~wq2_rptr[ASIZE], ~wq2_rptr[ASIZE-1], wq2_rptr[ASIZE-
                                    2:0]})
```
(3)

Inverting only the most-significant bit is the natural mistake, and it does not detect the wrap; in Gray code both the top bit and the one beneath it change when the counter passes the halfway point, so a single inversion detects that halfway crossing instead. This is the first fault seeded in Section 4.4.

Both flags are derived from the *next* pointer value rather than the registered one, so that they assert on the same clock edge on which the FIFO actually becomes full or empty, rather than one cycle later. A consequence of the synchroniser latency is that each domain observes a copy of the opposite pointer that is two clocks stale. The FIFO therefore reports itself fuller, or emptier, than it truly is. The error is always in the conservative direction and never the reverse, which is why the structure cannot overflow or underflow.

4.2 Randomised Scoreboard Testbench

A single directed test establishes very little about a CDC design. The testbench instead stalls both interfaces pseudo-randomly and runs the two clocks at a ratio the design was not written for. Two phases execute in one simulation: a fast writer against a slow reader (7 ns and 11 ns), then the reverse.

The choice of 7 and 11 is deliberate. Related periods such as 10 and 20 align their edges periodically and conceal precisely the class of defect being sought. A reference queue is pushed on the write side, and popped and compared on the read side.

Each phase terminates by halting writes and reading until the FIFO is empty, so that every word written is compared on the way out. Without this drain, several hundred words remain resident at the end of a phase and are never checked. Occupancy is computed from the design's own pointers,

$$\texttt{occupancy} = \texttt{wbin} - \texttt{rbin} \tag{4}$$

evaluated in `ASIZE+1` bit modular arithmetic. Counting writes and reads in the testbench instead would require comparing two quantities maintained on two different clocks, which disagrees by one whenever the two clock edges coincide; the first version of this testbench reported thousands of spurious overflows for exactly that reason.

4.3 Assertions and Functional Coverage

ModelSim ASE supports neither SVA nor covergroups, so the assertion layer executes under Vivado XSim against the same RTL and the same testbench. The checker is attached with `bind`, leaving the synthesisable source free of verification constructs. Table 4 lists the nine properties.

The most important is the Gray property itself, which tests the premise on which the entire design rests:

```
a_wgray_one_bit: assert property (
    @(posedge wclk) disable iff (!wrst_n)
    $countones(wgray ^ $past(wgray)) <= 1
);
```

Four covergroups record whether the interesting conditions were actually reached: the write and read interfaces crossed with their request signals, the occupancy distribution including both the empty and full bins, and concurrent activity on both interfaces. A test that passes without reaching full or empty has demonstrated nothing, so coverage is reported alongside every result.

4.4 Fault Injection

A test that has never failed is not evidence of anything. Before any result was accepted, defects were deliberately introduced into the design and the entire flow re-run to confirm that the verification detects them. Three faults were seeded; the outcomes are given in Table 6 and discussed in Section 5.3.

4.5 Skew Injection Experiment

The third seeded fault, removal of the Gray coding, was not detected. The explanation is that in RTL simulation every bit of a bus changes at the same instant, so the window in which a sampler could capture a mixture of old and new bits does not exist. The hazard is not modelled at this level of abstraction.

To expose it, a modified copy of the FIFO was built with two parameters: the encoding used on the crossing, and a per-bit transport delay applied to the crossing bus before it reaches the synchroniser. A counter records how many times a domain latched the bus while it was still settling, so that a clean run can be distinguished from one that never sampled at an unfavourable moment.

Two corrections were required before the experiment measured anything. First, the delay pattern must not be monotonic. Delaying bit i by i times a fixed step causes the low bits to move first, so the transient value is always smaller than both the old and the new pointer; that direction is harmless, because a pointer that reads low merely makes the opposite domain believe the FIFO is fuller, or emptier, than it is, which is the conservative direction the design already tolerates. The pattern used is proportional to $(3i+1) \bmod 4$, which permits a high bit to arrive before the low bits have cleared and the pointer to read transiently high. Second, the clocks must not be rationally related: with 7 ns and 11 ns a read edge falls only at whole-nanosecond offsets from a write edge, giving eleven fixed sampling phases which step over most of the sub-nanosecond windows of interest. The read clock was changed to 11.3 ns so that the phase drifts through all values, as two independent oscillators do.

4.6 Synthesis and CDC Checking

The design was synthesised out of context for the `xc7z020clg400-1` device. The objective is not a bitstream but three specific questions: whether any latches were inferred, whether the resource usage is consistent with intent, and whether the tool recognises both clock crossings.

The third question requires care. Vivado's `report_cdc` produces an empty report when no clocks are defined, and an empty report is easily mistaken for a clean one. Both clocks are therefore declared and placed in asynchronous clock groups before the report is generated.

4.7 Hardware Validation on PYNQ-Z2

The simulation testbench was rebuilt as synthesisable logic. A linear-feedback shift register generates the write data; an identical register on the read side predicts the expected value; a comparator counts mismatches. A second pair of registers stalls both interfaces so that the full and empty boundaries are reached. Counters are read back over JTAG through a VIO core.

If the FIFO were to drop or duplicate even a single word, the read-side replica would fall permanently out of step and every subsequent word would be counted as an error. A single-word failure therefore manifests as tens of millions of errors rather than one, which is the desired property of a checker.

Control values cross into the write domain through the design's own `sync_2ff` module. The stimulus rate settings are multi-bit and are held static while the test is stopped, so no multi-bit value is ever in transition when it is sampled; the counters are likewise read only after both sides have halted.

5. Results and Discussion

5.1 Functional Simulation

Table 3: Functional simulation results (ModelSim ASE 20.1)

Metric	Value
Clock ratios exercised	7/11 ns and 11/7 ns
Words written and compared	10,017
Data mismatches	**0**
Overflows / underflows	**0 / 0**
Words left unchecked at end of phase	**0**
Cycles with `wfull` asserted	12,678
Cycles with `rempty` asserted	12,419
Back-to-back writes	3,025
Concurrent read and write	3,546

Both boundary conditions were reached in quantity, which is what makes the zero mismatch count meaningful. A run that never filled or emptied the FIFO would report the same zero while having exercised none of the logic that matters.

5.2 Assertions and Coverage

Table 4: Assertions checked on the design

Property	Statement
`a_wgray_one_bit`	Write Gray pointer changes at most one bit per cycle
`a_rgray_one_bit`	Read Gray pointer changes at most one bit per cycle
`a_no_overflow`	Occupancy never exceeds the configured depth
`a_no_write_when_full`	Write pointer does not advance while full
`a_no_read_when_empty`	Read pointer does not advance while empty
`a_wptr_step`	Write pointer holds or advances by exactly one
`a_rptr_step`	Read pointer holds or advances by exactly one
`a_reset_not_full`	`wfull` is deasserted during reset
`a_reset_empty`	`rempty` is asserted during reset

Because occupancy is evaluated in modular arithmetic, `a_no_overflow` also detects underflow: were the read pointer ever to overtake the write pointer, the subtraction would wrap to a large value and violate the bound.

Table 5: Assertion and coverage results (Vivado XSim 2020.2)

Metric	Value
Words written and compared	10,015
Assertion failures across nine properties	**0**
Write interface coverage	100 %
Read interface coverage	100 %
Occupancy coverage, empty and full bins	100 %
Concurrent access coverage	100 %

The word counts differ by two between the simulators because the stimulus is randomised and the two implementations of `$urandom` do not generate identical sequences from the same seed.

5.3 Fault Injection Results

Table 6: Seeded faults and detection outcome

Seeded fault	Detected by
Full comparison with only the most-significant bit inverted	Occupancy check and scoreboard, within 500 ns of simulation time
The same fault, under XSim	`a_no_overflow` reported *occupancy 17 exceeds depth 16*
Gray coding removed, raw binary pointers crossing	**Not detected. The full testbench passed.**

The first two outcomes establish that the verification is capable of failing. The third is the substantive finding of this project and is developed below.

5.4 Skew Experiment Results

With the crossing switched to raw binary pointers, using the corresponding binary full and empty comparisons so that the encoding was the only variable, the complete testbench passed: zero mismatches, both clock ratios, both boundaries reached. The most frequently repeated claim about asynchronous FIFOs, that binary pointers across a clock boundary corrupt data, could not be reproduced by conventional verification.

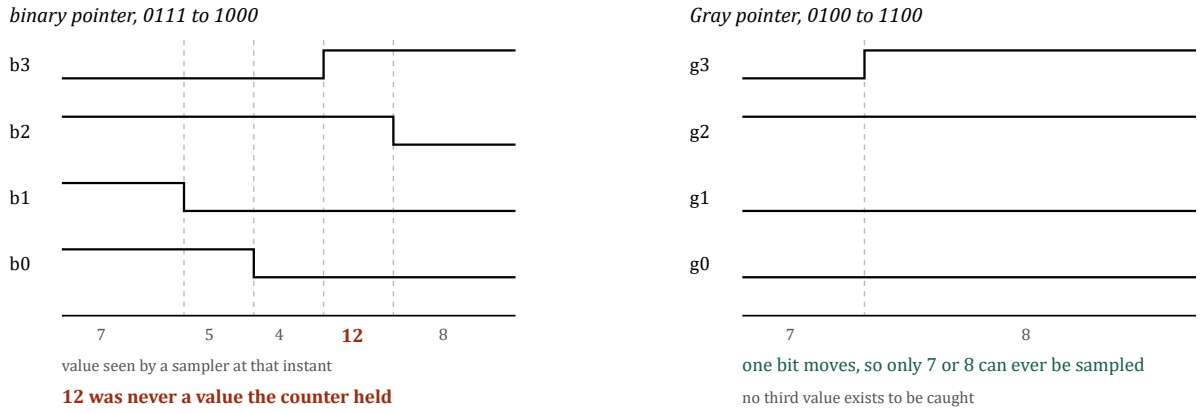

Figure 2: Why the encoding matters, drawn with the per-bit delay pattern used in the experiment. The binary pointer passes through 5, 4 and then 12 on its way from 7 to 8; a sampler that observes it during the third interval reads a value larger than either endpoint, which causes the opposite domain to believe transfers have occurred that have not. The Gray pointer moves a single bit, so no such value exists.

Table 7: Skew sweep results, 4,000 transactions per case, wclk 7.0 ns, rclk 11.3 ns

Skew (ns)	Gray captures	Gray errors	Binary captures	Binary errors
0.0	0	**0**	0	0
0.4	413	**0**	649	0
1.0	895	**0**	1,391	0
2.0	1,840	**0**	2,505	0
3.5	3,234	**0**	5,785	**3,513**

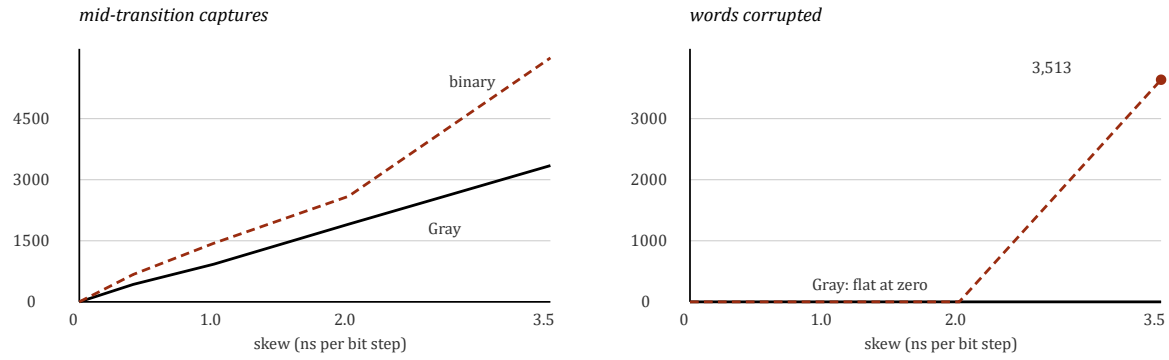

Figure 3: Skew sweep. Both encodings are sampled mid-transition increasingly often as skew rises (left), but only the binary encoding loses data, and only once the transient outlives a destination clock period (right).

Gray coding holds at zero errors across the entire sweep. In the final row it was captured mid-transition 3,234 times without corrupting or dropping a single word, because only one bit is ever in motion.

The binary result requires more explanation, since it survives the first three rows despite thousands of mid-transition captures. A transient exists only while a pointer is moving, and a pointer moves only when the domain that owns it has performed an operation. If the write domain latches a corrupted read pointer, the corruption is itself evidence that a read has just occurred, and therefore that a location has genuinely been freed. The corrupted value breaks the equality test for exactly one cycle, so at most one additional write is admitted, and that write had somewhere to go.

This self-limiting property holds only while the transient is shorter than a destination clock period, so that at most one sampling edge can fall inside it. At a step of 3.5 ns the spread across the bus is 10.5 ns against a 7 ns write clock; the corrupted value now survives across consecutive edges, several operations are admitted per event, and the argument collapses. Eighty-nine per cent of the words read were wrong.

Binary pointers are therefore not merely a riskier choice than Gray. They are correct only while skew remains below a bound that nobody specifies or checks, on a path that is by definition unconstrained, and they fail completely rather than gracefully once that bound is crossed. The guarantee provided by Gray coding is structural and has no such threshold.

5.5 Synthesis Results

Table 8: Out-of-context synthesis results (xc7z020clg400-1)

Metric	Value	Notes
Errors / warnings / critical warnings	**0 / 0 / 0**	Clean elaboration
Latches inferred	**0**	Explicitly checked
Slice LUTs	28	20 logic, 8 distributed RAM
Slice registers	40	All flip-flops
Block RAM / DSP	0 / 0	Memory in distributed RAM

The sixteen-by-eight memory was mapped to distributed RAM rather than a block RAM, which is expected at this size; a block RAM would be almost entirely unused.

Table 9: Vivado CDC report on the standalone FIFO

ID	Severity	Count	Description
CDC-3	Info	2	One-bit signal synchronised, ASYNC_REG present
CDC-6	Warning	2	Multi-bit bus synchronised, ASYNC_REG present

Both crossings were located, both at depth two, both covered by the asynchronous clock group exception. The CDC-6 warnings are correct and their absence would be more concerning than their presence: the tool observes a multi-bit bus entering a two-flop synchroniser, which is ordinarily a genuine defect, and nothing in the netlist informs it that this particular bus is Gray coded.

One unanticipated observation is that the report attributes the most-significant pointer bit to `wbin_reg[4]` rather than `wgray_reg[4]`. The top bit of a Gray code equals the top bit of the binary value from which it was derived, so Equation (1) leaves that bit unchanged and synthesis removed the duplicate register.

5.6 Hardware Validation Results

5.6.1 Clock Source Selection

The value of a hardware run lies almost entirely in the clocks, so that choice deserves justification. The convenient approach, deriving a second frequency from the board's 125 MHz oscillator using an MMCM, would have been considerably less work and would have demonstrated very little: two clocks produced by one MMCM are phase locked, the relationship between their edges is fixed and repeating, and sampling therefore occurs at a small number of relative phases indefinitely. Under such conditions a synchroniser may never once be exercised near its setup or hold boundary. The result would resemble a hardware test while proving less than the simulation already had.

Instead the write domain is clocked from `FCLK_CLK0`, generated by the processing system's PLL from the 33.333 MHz crystal, and the read domain from the independent 125 MHz board oscillator. The two sources share no reference and drift continuously with respect to one another, so over a ten-second run the sampling point sweeps through every phase relationship available. The processing system is present in the design for this reason alone; none of its other facilities are used.

5.6.2 Measured Results

Table 10: Hardware measurement on PYNQ-Z2, two phases of ten seconds

Phase	Words written	Words read	Errors
1 — write fast, read slow	159,905,237	159,905,221	**0**
2 — write slow, read fast	286,176,459	286,176,459	**0**
Reached full / empty			**yes / yes**
Worst setup slack (WNS)			1.997 ns
Worst hold slack (WHS)			0.061 ns

Phase 1 concludes sixteen words short of the number written, and the FIFO is exactly sixteen words deep. The phase terminates with the writer operating at its maximum rate against a slow reader, so the FIFO is full at the instant writes cease and is still holding a complete set of sixteen unread words. Phase 2 inverts the rates, the reader drains those words and thereafter keeps pace, and the two totals finish exactly equal. The residual is not measurement noise; it is the depth of the FIFO, and it corroborates that the design reached and held the full condition.

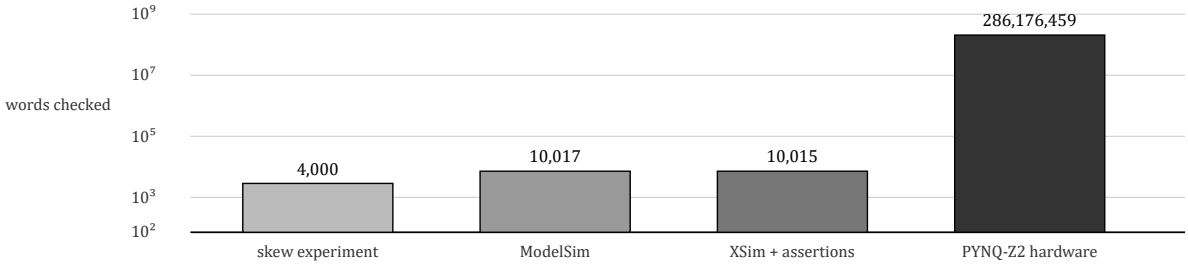

Figure 4: Words checked at each verification stage, logarithmic scale. The hardware run examined approximately 28,000 times as many transfers as the functional simulation, on real routing and with a continuously drifting phase relationship between the two clocks.

5.6.3 A Defect Found in the Test Harness

The first hardware build produced a CDC-10 violation, combinational logic before a synchroniser, on a path from the VIO output register to the asynchronous clear pin of a reset synchroniser flip-flop. The reset had been written as a single expression combining the processing-system reset with a software-controlled reset, and that gate sat directly on the clear input of the synchroniser.

The consequence, had it been left, is precise: a glitch on that gate resets one clock domain and not the other, leaving the FIFO's two pointers disagreeing about where valid data begins. This is the exact failure mode the design exists to prevent, occurring inside the harness constructed to demonstrate that it does not occur. The reset was restructured into three stages, with a single asynchronous source carrying no logic in front of it, the software reset crossing through a `sync_2ff` instance, and the combination registered before distribution.

The full-design report also raises thirty-two CDC-1 criticals, all on paths from the FIFO memory to the comparator. These are correct as they stand. Data is written on one clock and read on another with no synchroniser between, which is what a CDC checker should report, except that the pointer protocol guarantees the read side addresses only words written at least two synchroniser stages earlier; the data has been stationary for several clock periods before anything reads it. The tool analyses connectivity, not protocol. Notably these did not appear in the out-of-context run of Section 5.5, because there `rdata` terminates at a port with no consumer logic to trace.

5.7 Limitations

The skew experiment models bus skew and not metastability. Its synchroniser consists of two ideal flip-flops which resolve instantaneously, so the experiment says nothing about MTBF, and no quantity of Gray coding removes the requirement for the synchroniser. The two mechanisms address different failure modes and only one of them is exercised there.

The hardware measurement is likewise not a reliability measurement. Twenty seconds of two drifting clocks constitutes a thorough sweep of the phase space, but a two-flop synchroniser has a finite failure rate which a run of this duration would not be expected to expose. Nothing reported here justifies two synchroniser stages in preference to three at a higher operating frequency; that determination requires an MTBF calculation rather than a test.

Table 11: Consolidated verification summary

Stage	Tool	Words checked	Outcome
Functional simulation	ModelSim ASE 20.1	10,017	**0 mismatches**
Assertions and coverage	Vivado XSim 2020.2	10,015	**0 failures, 100 % coverage**
Fault injection	both	—	2 of 3 faults detected
Skew experiment	ModelSim ASE 20.1	4,000	Gray 0, binary 3,513 errors
Synthesis	Vivado 2020.2	—	**0 latches, 0 warnings**
Hardware	PYNQ-Z2, Zynq-7020	286,176,459	**0 errors**

6. Conclusion

1. **The design is correct across every level tested.** Ten thousand words in simulation with zero mismatches, nine assertions with no failures and complete functional coverage, synthesis with no inferred latches and no warnings, and 286,176,459 words on hardware with zero errors while reaching both the full and empty boundaries.
2. **Verification must be shown capable of failing.** Two seeded faults were detected promptly, one by the scoreboard within 500 ns and one by the `a_no_overflow` assertion. Without that demonstration the passing results would carry substantially less weight.
3. **RTL simulation cannot justify the Gray coding.** Removing it entirely produced no failure, because every bit of a bus changes at the same instant in simulation and the hazard therefore does not exist to be found. This is the most useful thing the project established, and it is a limitation of the method rather than of the design.
4. **Gray coding provides a structural guarantee, not a margin.** Under injected skew it held at zero errors while being captured mid-transition thousands of times. Binary pointers survived small skew for an identifiable reason, then failed at eighty-nine per cent corruption once the transient outlived a destination clock period.
5. **Tool reports require interpretation rather than compliance.** The CDC-6 warnings on the Gray buses are expected and their absence would indicate a problem; the thirty-two CDC-1 criticals on the memory path are correct because the pointer protocol, which the tool cannot see, keeps the data stationary. The CDC-10 violation, by contrast, was a real defect and was fixed.
6. **Independent clock sources are what make a hardware run meaningful.** Driving both domains from one MMCM would have been simpler and would have left the synchroniser potentially unexercised. Two crystals with no common reference sweep the entire phase relationship continuously.

7. Future Work

1. **MTBF characterisation of the synchroniser.** The present work demonstrates correct operation but measures no failure rate. Deliberately reducing the settling time available, by raising the destination clock frequency or inserting a controlled delay, would allow synchroniser failures to be observed and the classical exponential MTBF model to be fitted against measurement.
2. **Gate-level simulation with back-annotated delays.** The skew in Section 4.5 is injected by hand. Running the post-implementation netlist with SDF timing would exercise the real routing delays of the placed design and remove the need to choose a skew pattern.
3. **Formal property verification.** The nine assertions are currently checked only over the stimulus the testbench happens to generate. Proving them exhaustively with a model checker would establish the pointer and flag properties for all reachable states rather than for those visited.
4. **An AXI-Stream wrapper.** Presenting the FIFO through a standard ready/valid interface would make it directly usable in the M.Tech accelerator work and would allow the interface protocol itself to be covered by assertions.
5. **Parameter sweep and depth generalisation.** All results reported here use a single configuration. Regression across data widths and depths, and an examination of what breaks when the depth is not a power of two, would establish the limits of the parameterisation.

References

[1] C. E. Cummings, “Simulation and synthesis techniques for asynchronous FIFO design,” in *Proc. Synopsys Users Group Conference (SNUG)*, San Jose, USA, 2002.

[2] R. Ginosar, “Metastability and synchronizers: A tutorial,” *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, Sep./Oct. 2011.

[3] R. Ginosar, “Fourteen ways to fool your synchronizer,” in *Proc. 9th IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC)*, Vancouver, Canada, pp. 89–96, 2003.

[4] W. J. Dally and J. W. Poulton, *Digital Systems Engineering*. Cambridge, UK: Cambridge University Press, 1998.

[5] F. Gray, “Pulse code communication,” U.S. Patent 2,632,058, Mar. 17, 1953.

[6] C. E. Cummings and P. Alfke, “Simulation and synthesis techniques for asynchronous FIFO design with asynchronous pointer comparisons,” in *Proc. Synopsys Users Group Conference (SNUG)*, San Jose, USA, 2002.

[7] *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[8] Xilinx Inc., “Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906),” Xilinx Technical Documentation, 2020.

[9] Xilinx Inc., “Vivado Design Suite User Guide: Using Constraints (UG903),” Xilinx Technical Documentation, 2020.

[10] Xilinx Inc., “Zynq-7000 SoC Data Sheet: Overview (DS190),” Xilinx Technical Documentation, 2021.

[11] AMD/Xilinx, “PYNQ: Python productivity for Zynq — framework documentation,” 2022. [Online]. Available: http://www.pynq.io